

Experiment One

Object Detection and Recognition

Training and deploying a mouse-and-cup detector on Jetson

Final model	YOLO11n version 13
Target classes	mouse and cup
Training input	768 by 768 pixels
Deployment	Windows ONNX and Jetson PyTorch CUDA
Experiment period	August to September 2026

Submission note All numerical claims in this report are tied to files stored in the project workspace. The older CUP3 clip is excluded from the final evidence because it used an earlier unsuccessful test.

Executive Summary

This project started as a simple mouse-and-cup detector, but the first live tests exposed problems that were not obvious from the training numbers. Black phones and robot parts were sometimes called mice, side-view mice were missed, and the cup only worked reliably while upright. I kept those failures and used them to decide what to collect next. The final model is YOLO11n v13, trained at 768 pixels and exported as both PyTorch and ONNX weights.

The final export contains 793 images: 492 for training, 61 for validation and 240 for testing. The training split includes 49 empty label files on purpose. They show phones, chair backs, robot shells, laptop edges and other objects that caused false mouse detections. Version 13 also adds eighty manually checked frames, with most of the new positive examples covering cups lying on their side and mouse poses that had previously failed.

I checked twenty different viewing conditions for the classroom test. A scene only counted as correct when every expected object was found and there was no extra box. Eighteen of the twenty scenes passed, so the final score is 90 percent. Eighteen scenes are taken from Jetson recordings. The other two are horizontal-cup frames replayed through the final v13 ONNX model because the older CUP3 clip was upright and did not detect the cup.

The selected Jetson frames show 15.7 to 21.2 FPS, averaging about 17.9 FPS. The ROS 2 test was also run on the physical board: /dev/video0 produced a mouse detection at 0.9478 confidence, and /yolo/detections was measured at about 20.57 Hz. The live display and the ROS 2 message therefore came from the same working detection path.

Acceptance Results



Accuracy uses strict scene scoring all expected targets present and no extra false box

Figure 1 Summary of the measured acceptance results

1 Requirements and Evidence

Before preparing the final files, I copied each item from the assessment sheet into a checklist. The table below records what I used as evidence for each item. This caught one missing item early: an upright cup video could not be used to prove that a cup lying flat was recognised.

Requirement	Evidence	Status
Two desktop classes	Class IDs 0 mouse and 1 cup in dataset.yaml	Met
Self collected and labelled data	Audited v13 dataset and X AnyLabeling batches	Met
Jetson execution	Nine v13 Jetson result clips and CUDA deployment script	Met
Class box and confidence overlay	Rendered v13 videos and live inference code	Met
Twenty angles at 80 percent	18 correct scenes from 20 selected scenes	Met at 90 percent
At least 5 FPS on Jetson	15.7 to 21.2 FPS in selected Jetson frames	Met
Save results and errors	Annotated and raw video outputs plus review frames	Met
ROS 2 detection publishing	Jetson camera Detection2DArray, mouse 0.9478, about 20.57 Hz	Met

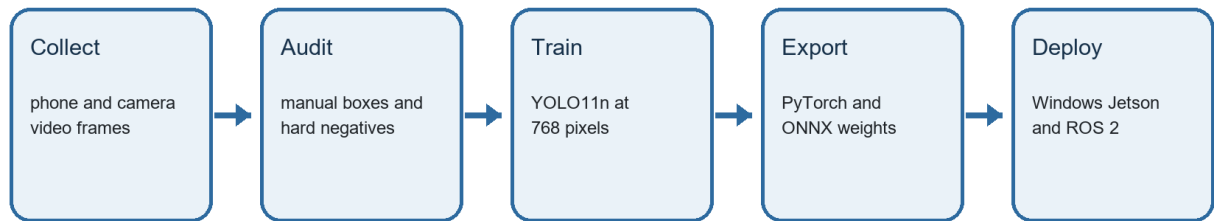
The package first passed a recorded-video replay in WSL, which checked message construction and DDS publication. I then repeated the full path on the physical Jetson with its camera. The final evidence package contains the annotated AVI, a representative frame, the detector log and a non-empty topic record. Keeping both runs makes the software test and the hardware acceptance test independently reproducible.

- Accuracy is scene based because the teacher accepted twenty angles rather than twenty unique object instances.
- The horizontal cup evidence is included as an explicit supplement and not mixed into the Jetson FPS calculation.
- The old CUP3 recording is intentionally ignored because it showed an upright stainless cup with no successful detection.

2 System Design

My working cycle was: record a short video, keep representative frames, correct the boxes in X AnyLabeling, run the audit, export the YOLO folders, train, and test again with the camera. When the model made a mistake, I saved that frame. A missed object became a positive sample after I drew the box; a false alarm became an empty-label negative sample.

End to End Detection Workflow



Feedback loop missed detections and false alarms become the next audited batch

Figure 2 Data training deployment and error feedback workflow

This workflow explains why the project progressed through several versions. Each version addressed a specific failure rather than changing the model architecture at random. Versions 9 and 10 improved camera domain coverage. Version 11 added frames taken from missed and false detections. Version 12 included phones as difficult negative examples because phones were often predicted as mice. Version 13 concentrated on horizontal cups and additional poses.

The output side has two parallel paths. The Windows path uses ONNX Runtime and is convenient for camera testing and data capture. The Jetson path uses the PyTorch best.pt weight with CUDA through Ultralytics. The ROS 2 node wraps the Jetson prediction result in standard vision_msgs structures so other robot nodes can consume the class, confidence and bounding box without parsing screen graphics.

3 Development and Deployment Environment

Component	Development computer	Jetson deployment
Operating system	Windows 11 and WSL 2 Ubuntu 22.04	Ubuntu 22.04 with JetPack 6.2.1
Python	3.10.12 training and 3.13 ONNX	Jetson Python environment
Deep learning	PyTorch 2.12.1 plus CUDA 12.6	PyTorch 2.5.0a0 Jetson build
Detection framework	Ultralytics 8.4.127	Ultralytics 8.4.128
Image processing	OpenCV 4.10	OpenCV 4.13
ROS	ROS 2 Humble in WSL	ROS 2 Humble with vision_msgs
Camera	USB camera index 1	V4L2 camera device

WSL 2 was useful because Ultralytics training and ROS 2 Humble both behaved like a normal Ubuntu installation while the dataset remained on the Windows F drive. Training paths in args.yaml therefore appear as /mnt/f paths. The Windows ONNX program uses camera index numbers, while Jetson normally exposes the same hardware as /dev/video0 or /dev/video1.

I used environment checks before long training jobs. The checks confirm that Python can import OpenCV, PyTorch and Ultralytics, that CUDA is visible, and that ROS 2 commands are sourced correctly. An environment check is not a substitute for a task test. For example, publishing a test string proves DDS communication but does not prove that object detection messages contain correct boxes. This distinction led to the dedicated detector node added later.

```
python scripts/check_env.py
python scripts/check_yolo.py
bash scripts/check_ros2.sh
```

4 Object Classes and Annotation Policy

The class set is intentionally small. Class 0 is mouse and class 1 is cup. Both are common desktop objects, but their shapes are not equally easy. A cup is often tall and visually distinctive. A mouse can be small, dark and similar to phones, power banks, chair parts or robot components. The label policy therefore needs consistency more than complexity.

ID	Class	Box rule	Important edge cases
0	mouse	Tight box around the visible mouse body	Side view underside hand occlusion and small distant views
1	cup	Tight box around the visible cup body	Horizontal pose partial occlusion lid and reflective surfaces

When two target objects overlap, both are labelled if each remains meaningfully visible. This is not duplicate annotation. A duplicate exists only when two boxes describe the same instance. Earlier audits found no same class duplicate box with intersection over union above 0.5. Empty label files are valid when no mouse or cup appears; they teach the model that a visually similar black object is background.

I reviewed model generated prelabels manually in X AnyLabeling. Prelabels accelerated the first pass, but they were never treated as ground truth automatically. I corrected missing objects, removed boxes around phones and robot bodies, and checked that boxes did not extend outside the image. Two v13 boxes near image borders were clipped during export, and the audit report contains no fatal errors.

5 Data Collection Strategy

At first I extracted too many neighbouring frames from phone videos. The image count increased, but many frames showed almost the same pose and background. In the later sessions I collected fewer, more useful differences: front and side views, high and low camera positions, rotation, hand occlusion, distance changes, new desks and the black objects that had triggered false boxes.

The horizontal cup session was recorded specifically after the assessment requirement changed. It includes a cup lying fully on its side, movement across the table and several backgrounds. Eighty representative frames were selected and manually reviewed for version 13. The original raw video was kept so that the final model could later be replayed on exactly the captured sequence.

- Positive diversity included upright cups, horizontal cups, blue and green cups, mouse top views, side profiles and underside views.
- Hard negatives included phones, laptop edges, screens, chairs, bags, robot bodies, cables and other black objects.
- Raw and annotated videos were both saved where practical so wrong predictions could be reproduced without filming again.
- Frames from one continuous scene were not relied on as independent proof of generalisation.

The final twenty angle test is separate from the training set audit. It is a practical demonstration rather than a substitute for the held out test partition. Both forms of evaluation are needed: the image split produces repeatable numerical metrics, and the live sequence shows whether the program behaves acceptably during actual use.

6 Dataset Cleaning and Audit

The audit found several ordinary labelling mistakes. `hard_robot_mouse` had a misleading filename but no real mouse, so I kept it as a negative. `neg_laptop` did contain part of a cup at the left edge, so I added that cup box. I also found that the first export script silently treated `train_candidate` and `not_assigned` as training data. I changed the script so only explicit `train`, `val` and `test` rows are exported.

The audit process performs structural checks before training. It verifies that every exported image has one matching label file, class IDs are valid, normalised coordinates are within range, box width and height are positive, and the train validation and test lists do not contain the same file. It also produces an exclusion list so rejected or unassigned samples remain visible for later review rather than disappearing.

Audit check	Reason	v13 outcome
Image label pairing	Prevents unintentional missing annotations	Passed
Class ID range	Only mouse and cup are valid	Passed
Coordinate range	YOLO values must remain between zero and one	Passed after two clips
Empty labels	Retains deliberate hard negatives	58 in audit and 55 in exported splits
Split membership	Avoids hidden train candidate promotion	Explicit train val test only
Fatal audit errors	Stops training on invalid data	None

7 Version 13 Dataset Composition

The authoritative exported directory contains 793 images. Parsing the label files directly gives 635 mouse boxes and 422 cup boxes. There are 55 empty labels across the three splits. The counts below are calculated from the actual exported files rather than copied from a planning note.

Split	Images	Mouse boxes	Cup boxes	Empty labels
train	492	402	273	49
val	61	59	33	0
test	240	174	116	6
Total	793	635	422	55

Version 13 Exported Image Splits

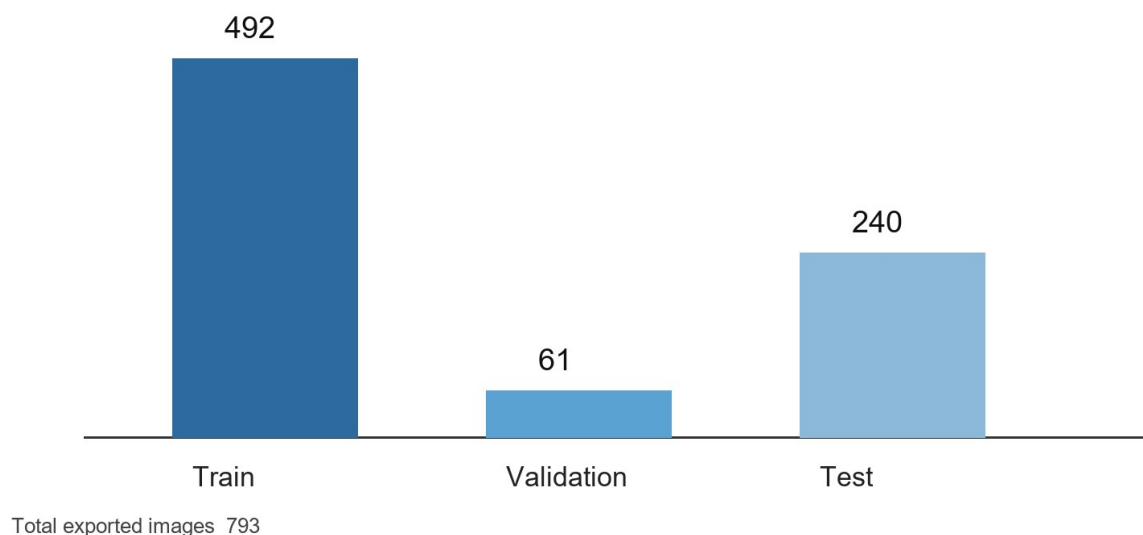


Figure 3 Exported train validation and test image counts

The test partition is relatively large because it preserves independent original scenes and is reused for comparison between model versions. The training partition is smaller than the raw audit store because 461 audit records are explicitly excluded. This is preferable to increasing the count with uncertain or duplicate samples.

Dataset Sample Gallery

These are actual images from the exported v13 folders. The boxes are drawn from the matching YOLO label files, not from model predictions. I selected them to show the cases that mattered during development: a normal mouse, its underside, a cup lying flat, a blurred two-object recovery sample, a validation scene and a phone kept as background.

Examples from the Audited Version 13 Dataset

Green = mouse Blue = cup Grey = deliberate background sample



Plate 1 Real v13 images with audited ground-truth boxes

The phone card has no box because it is a deliberate hard negative. This is how the dataset tells the model that a dark rectangular object is not automatically a mouse. The blurred card is included because it came from a real failure, not because it is visually clean.

8 Dataset Version History

Version	Primary change	Reason
v7	First audited baseline at 768 pixels	Establish repeatable training
v8	Cleaned export and evaluation scripts	Improve split reliability
v9	External camera data	Reduce camera domain mismatch
v10	Manual camera reannotation	Improve mouse and cup angle coverage
v11	Live error feedback samples	Correct misses and false positives
v12	Phone hard negatives and mouse views	Stop phone to mouse confusion
v13	Horizontal cup and new environments	Meet the horizontal cup requirement

The version number increased only after a new data batch or a specific correction. The metrics did not rise every time. In fact, v13 has slightly lower held-out mAP than v12, but it recognises the horizontal cup pose that v12 lacked. I kept v13 for the final demonstration because that pose was part of the teacher's requirement and the twenty-angle test still passed.

Keeping separate version folders and training scripts makes the experiment reproducible. It also prevents a good weight file from being overwritten by a later trial. Each training output has args.yaml, results.csv, plots and best and last weights. This structure allowed me to trace exactly which dataset and starting weight produced the final model.

9 Model Choice and Architecture

I selected YOLO11n because the experiment values real time operation on Jetson more than maximum offline accuracy. The nano variant is small enough to run comfortably above 5 FPS while still learning two simple classes. A one stage detector predicts boxes and class scores in one network pass, which is simpler to deploy than a two stage system.

The model input is a tensor with batch, channel, height and width dimensions. Camera frames arrive in HWC layout using OpenCV BGR channel order. During inference the program letterboxes the image to 768 by 768, converts BGR to RGB, transposes HWC to CHW, normalises byte values to floating point zero to one, and adds a batch dimension. ONNX Runtime then returns candidate box coordinates and class scores.

ONNX is an interchange format for neural networks. It separates the trained computation graph from the original training program, so the Windows test tool can run the model with ONNX Runtime without importing the full Ultralytics training stack. The Jetson path keeps the PyTorch best.pt weight because Ultralytics can use CUDA directly and the result object is convenient for ROS 2 message construction.

Artifact	Purpose	Approximate role
best.pt	Jetson and Ultralytics inference	Full PyTorch checkpoint
best.onnx	Portable Windows inference	Exported computation graph
dataset.yaml	Class names and split locations	Training data entry point
args.yaml	Exact training configuration	Reproducibility record

10 Version 13 Training Configuration

Parameter	Value	Interpretation
Initial weight	v12 best.pt	Continue from the previous cleaned detector
Epoch limit	40	Maximum complete passes through training data
Patience	12	Stop when validation improvement stalls
Batch	16	Images processed before each optimiser update
Image size	768	Square network input resolution
Optimiser	SGD	Momentum based gradient update
Initial learning rate	0.00015	Small fine tuning step
Final rate factor	0.01	Final rate relative to initial rate
Momentum	0.937	Smooths gradient direction
Weight decay	0.0005	Regularises network weights
Seed	20260824	Makes the split order and randomness repeatable
Mixed precision	enabled	Reduces GPU memory and improves speed

SGD was chosen for controlled fine tuning. The very small initial learning rate is appropriate because v13 starts from an already trained v12 model. A larger learning rate could destroy useful features while fitting only the new horizontal cup batch. Training used RAM caching and four data workers in WSL on the laptop GPU.

```
cd /mnt/f/PycharmProjects/robomaster
bash scripts/train_v13_horizontal_cup_wsl.sh
```

11 Loss Functions and Data Augmentation

The detector uses three main loss terms. The box loss weight is 7.5 and penalises incorrect localisation. The classification loss weight is 0.5 and separates mouse from cup and background. The distribution focal loss weight is 1.5 and helps refine bounding box edges. These values came from the actual v13 args.yaml file.

Augmentation	Value	Effect
Hue shift	0.015	Small colour change
Saturation	0.7	Wide colour intensity variation
Brightness	0.4	Lighting variation
Translation	0.1	Moves targets within the frame
Scale	0.5	Changes apparent object size
Horizontal flip	0.5	Mirrors half of training samples
Mosaic	1.0	Combines several images early in training
Random erasing	0.4	Simulates partial occlusion
Rotation shear perspective	0	Not enabled in this run
MixUp CutMix Copy Paste	0	Not enabled in this run

Augmentation changes image appearance and placement, but it cannot create a truly new three dimensional viewpoint. This was an important lesson from the mouse failures. Rotating a top view by twelve degrees is still a top view; it does not become a side or underside image. Real collection remained necessary even after aggressive augmentation.

12 Training Results

The best validation mAP50 to 95 occurred at epoch 27. At that point precision was 0.856, recall was 0.776, mAP50 was 0.886, and mAP50 to 95 was 0.715. I used best.pt for deployment instead of assuming that the last epoch was best.

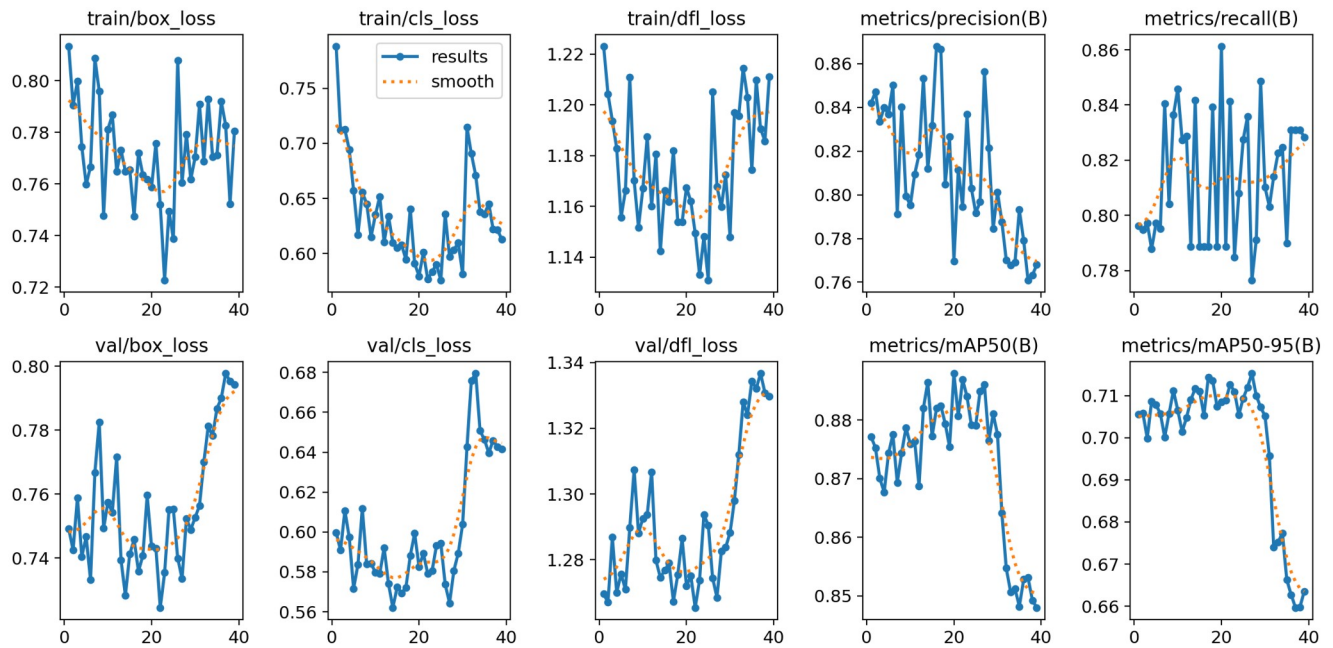


Figure 4 Version 13 training and validation curves

Figure 4 is the original results.png written by Ultralytics during the v13 run; it has not been redrawn for this report. The left-side plots show box, classification and DFL losses, while the right-side plots show precision, recall and mAP. The validation curves worsen after the best region even though some training losses continue to fall. That is why I kept the checkpoint from the best validation epoch. More independent horizontal-cup scenes would be more useful than simply adding epochs.

13 Validation Analysis

The normalised confusion matrix gives a class focused view. About 0.80 of true mouse instances are classified as mouse, while about 0.85 of true cup instances are classified as cup. Background related false positives remain visible, especially for mouse. This matches live testing: small dark objects are more likely to be confused with a mouse than with a cup.

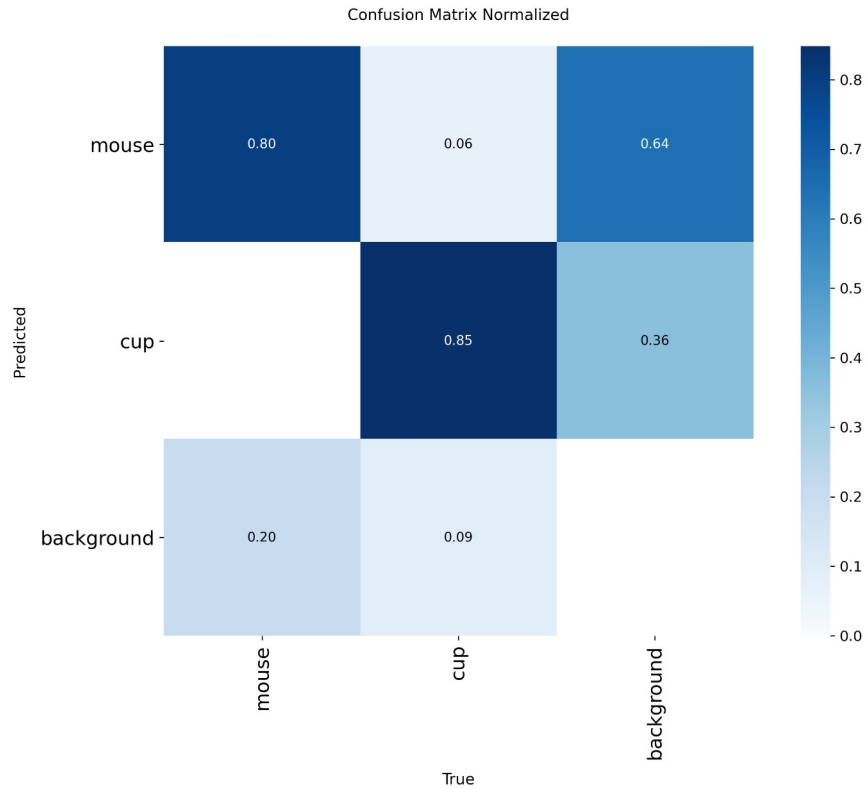


Figure 5 Normalised validation confusion matrix for version 13

The held out test partition is intentionally harder than the classroom demonstration. At the same test configuration, v13 produced precision 0.706, recall 0.635, mAP50 0.677 and mAP50 to 95 0.374. Version 12 had slightly better general test metrics. I therefore do not claim that version 13 is universally superior. Its value is the targeted horizontal cup behaviour combined with the practical 90 percent acceptance result.

14 ONNX Inference Pipeline

The Windows program performs preprocessing, inference, postprocessing and display in a continuous loop. Letterboxing preserves the source aspect ratio while filling unused pixels with a neutral grey value. The original scale and padding are retained so output boxes can be mapped back to camera coordinates. Non maximum suppression is performed separately for each class to remove duplicated overlapping predictions.

```
letterboxed, scale, pad_x, pad_y = letterbox(frame, imgsz)
input_tensor = cv2.cvtColor(letterboxed, cv2.COLOR_BGR2RGB)
input_tensor = input_tensor.transpose(2, 0, 1).astype(np.float32) / 255.0
input_tensor = np.expand_dims(input_tensor, axis=0)
output = session.run(None, {input_name: input_tensor})[0]
```

Independent confidence thresholds are useful because the two classes have different error patterns. Mouse errors were usually false positives on dark objects, so a high mouse threshold reduces those mistakes. Cup predictions were generally cleaner. The final demonstration used 0.75 for both classes to make the scoring rule conservative.

A short temporal smoother requires two matching frames before displaying a new detection and can hold a track for three missed frames. This reduces flicker. It does not create a detection from nothing, and it should not be used to inflate still image accuracy. For that reason the practical evidence is evaluated scene by scene rather than counting every held display frame as a new success.

15 Live Display and Video Recording

The live application overlays each accepted class name, confidence value and coloured rectangle. Mouse boxes are green and cup boxes are blue. A status line shows smoothed FPS, current detection count and class thresholds. Pressing Q or Escape ends the loop and releases the camera and video writer cleanly.

```
py -3.13 scripts/live_camera_onnx.py `
--camera 1 `
--model runs/detect/mouse_cup_yolo11n_v13_horizontal_cup_768/weights/best.onnx `
--mouse-conf 0.75 --cup-conf 0.75 `
--save results/v13_camera1_detected.avi `
--save-raw results/v13_camera1_raw.avi
```

Saving both streams serves different purposes. The annotated file is evidence for the assessor because it contains boxes, labels, confidence and FPS. The raw file preserves the actual camera input and is more valuable for debugging. It can be replayed through a later model version without repeating the physical experiment.

The new run_video_onnx.py utility follows the same preprocessing and class thresholds for recorded input. I used it to replay the horizontal cup capture through the final v13 ONNX model and produce a ten second supplemental clip. The JSON summary records the source, model, selected time range, thresholds and number of frames containing each class.

16 Jetson Deployment

On Jetson, the program uses the v13 best.pt file and CUDA. OpenCV opens the V4L2 camera, Ultralytics performs prediction, and the application draws the same information as the Windows version. The Jetson recordings are 640 by 480 MJPEG at 20 FPS. This camera rate is appropriate for the experiment and keeps end to end latency low.

```
python3 scripts/live_camera_pt.py \
  --camera 0 \
  --model /home/nvidia/jetson_yolo/best.pt \
  --imgsz 768 \
  --mouse-conf 0.75 --cup-conf 0.75 \
  --save /home/nvidia/jetson_yolo/results/v13_jetson_detected.avi
```

The selected evidence frames report a minimum of 15.7 FPS and a maximum of 21.2 FPS, with an average near 17.9 FPS. Even the minimum is more than three times the required 5 FPS. Because the horizontal supplement was processed offline on Windows, those two frames are excluded from the Jetson speed average.

Measurement	Observed	Requirement	Result
Selected Jetson minimum	15.7 FPS	5 FPS	Pass
Selected Jetson maximum	21.2 FPS	5 FPS	Pass
Selected Jetson mean	about 17.9 FPS	5 FPS	Pass
Camera stream	640 by 480 at 20 FPS	real time	Pass

17 Twenty Angle Test Protocol

The teacher confirmed that twenty angles are sufficient. I selected twenty representative moments rather than claiming twenty different physical object models. The set covers upright, tilted and horizontal cups, two cups together, two classes together, mouse top views, side profiles and underside views.

A scene is correct only when all expected targets are detected and no extra false positive appears. This is stricter than target recall alone. A two object scene fails if one target is missing. The method also prevents a long easy clip from dominating the result because every selected angle contributes one score.

1. Choose a timestamp with a clear and stable object pose.
2. Record the expected target list before reading the prediction.
3. Check the rendered frame for every expected box and for extra boxes.
4. Mark the whole scene correct or incorrect using the strict rule.
5. Count correct scenes and divide by twenty.

The two horizontal cup angles replace redundant upright cup views. The old CUP3 file is excluded. This makes the evidence closer to the actual assessment requirement and removes a clip that could be misinterpreted as proof of a successful horizontal cup detection.

18 Twenty Angle Results

No	Expected	View	Platform	Result
1	cup	upright front	Jetson	Pass
2	cup	upright alternate side	Jetson	Pass
3	cup	horizontal right-facing	offline v13 ONNX	Pass
4	cup	horizontal shifted view	offline v13 ONNX	Pass
5	cup	tilted diagonal	Jetson	Pass
6	cup	blue cup front	Jetson	Pass
7	cup	blue cup alternate side	Jetson	Pass
8	cup	blue cup offset	Jetson	Pass
9	cup+cup	two cups together	Jetson	Pass
10	mouse+cup	two classes together	Jetson	Pass
11	mouse	left side	Jetson	Pass
12	mouse	front diagonal	Jetson	Pass
13	mouse	low side profile	Jetson	Pass
14	mouse	top diagonal	Jetson	Pass
15	mouse	rotated top	Jetson	Pass
16	mouse	right side	Jetson	Pass
17	mouse	underside horizontal	Jetson	Pass
18	mouse	underside diagonal	Jetson	Pass
19	mouse	extreme side profile	Jetson	Miss
20	mouse	underside	Jetson	Miss

Eighteen of the twenty scenes are correct, giving 90 percent strict scene accuracy. The two failures are mouse views: an extreme side profile and an underside pose. No selected scene failed because of a wrong class label. These failures are retained because a credible experiment should show both successful and unsuccessful cases.

19 Evidence Frames One to Ten

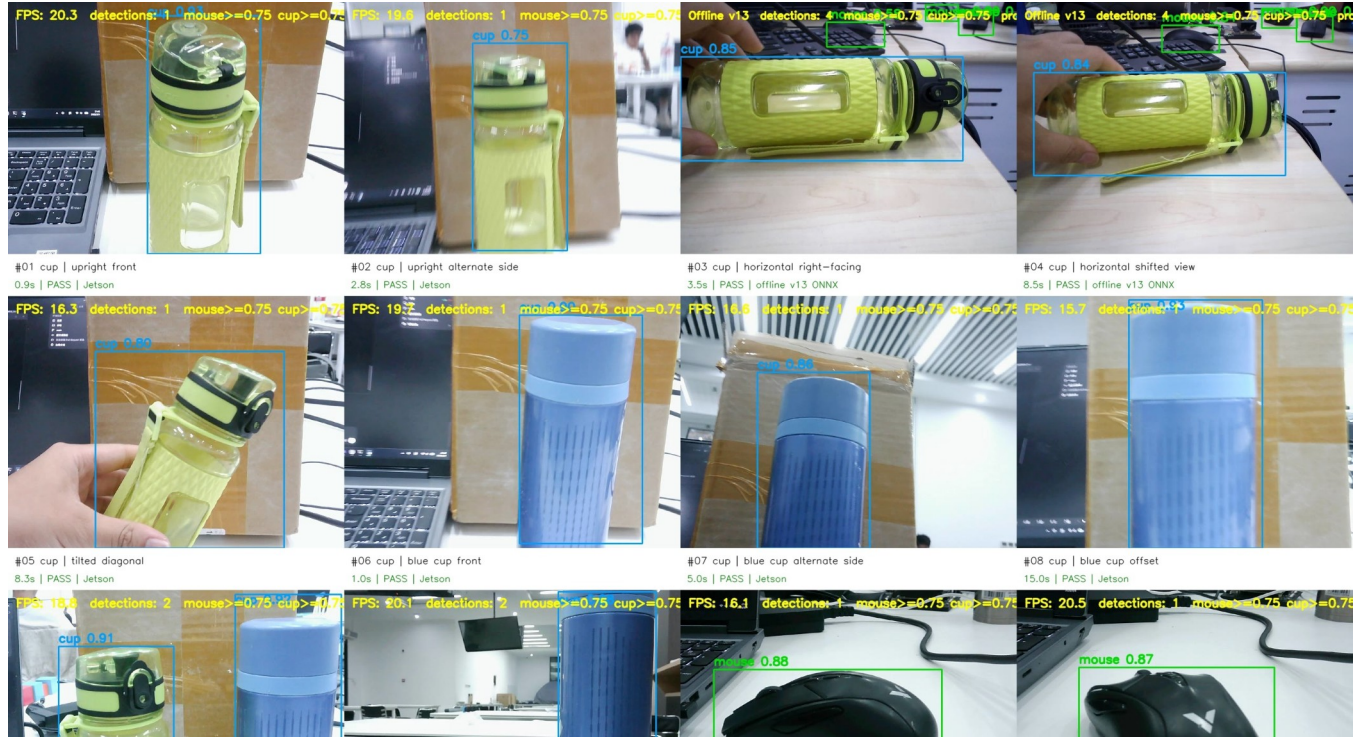


Figure 6 Cup and mixed class evidence for test scenes 1 to 10

Scenes 1, 2 and 5 show the green cup upright and tilted. Scenes 3 and 4 show the same target fully horizontal with v13 ONNX confidence around 0.84 to 0.85. Scenes 6 to 8 use a blue cup. Scene 9 contains two cups, and scene 10 proves simultaneous mouse and cup detection.

The first two rows use Jetson output except for the two explicitly labelled offline v13 frames. The platform label is printed below each evidence image so the supplemental replay cannot be confused with Jetson execution.

20 Evidence Frames Eleven to Twenty

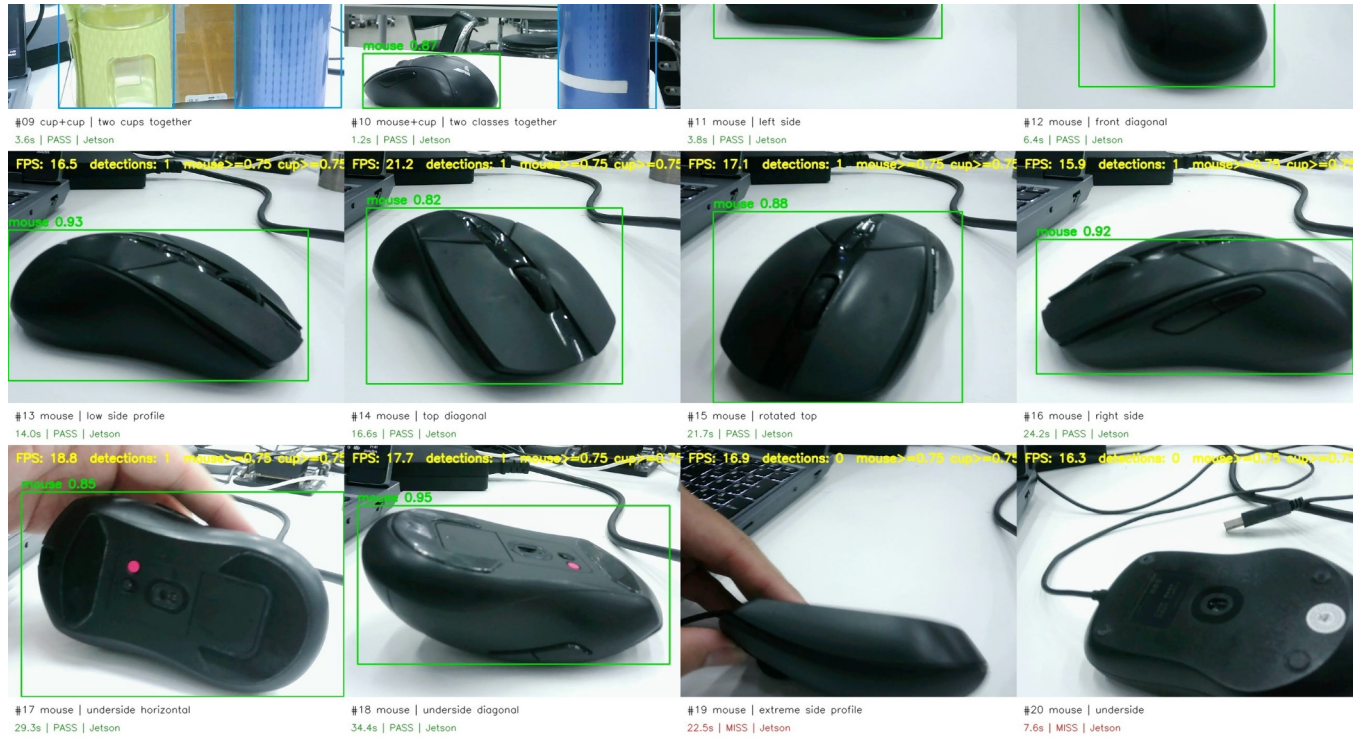


Figure 7 Mouse evidence and retained failure cases for test scenes 11 to 20

Scenes 11 to 18 show successful mouse detections across side, diagonal, top and underside views. Scenes 19 and 20 are the two misses. Their inclusion explains why the final score is 90 rather than 100 percent and identifies the most useful direction for future data collection.

All ten frames on this page come from Jetson recordings. The displayed FPS values range from 15.9 to 21.2 in the successful frames and remain above 16 FPS in the two misses, confirming that the failures are recognition errors rather than a stalled inference loop.

21 Horizontal Cup Supplement

The previous CUP3 clip did not satisfy the assessment need. It showed an upright stainless cup and the model did not recognise it. I therefore ignored that clip and used the separate v13 horizontal cup result already stored in the results directory. The final v13 ONNX model was replayed for thirteen seconds with both class thresholds set to 0.75.

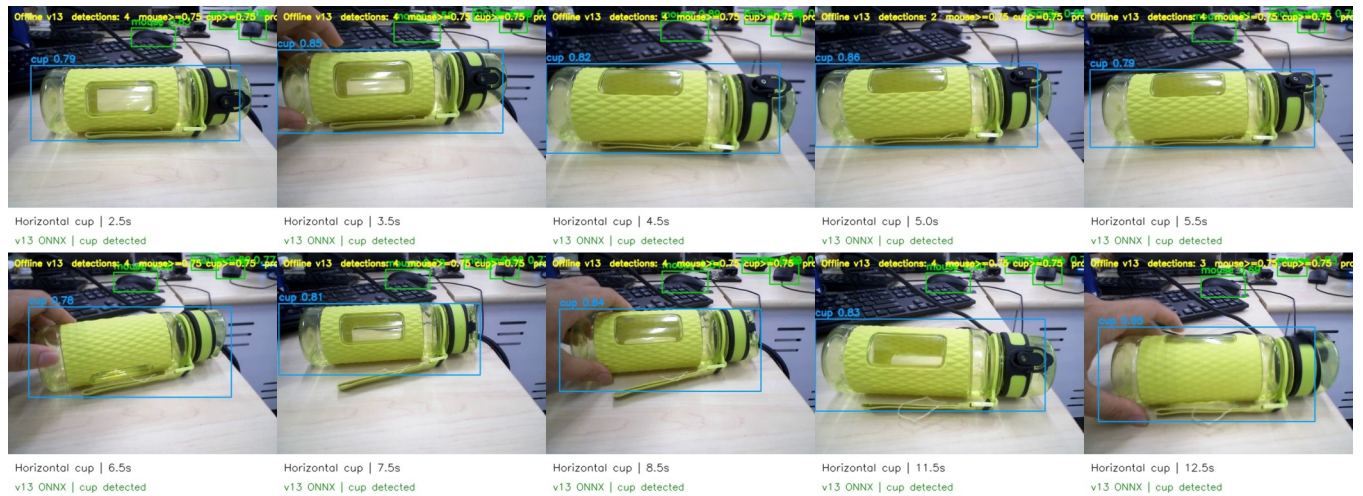


Figure 8 Thirteen second horizontal cup replay through the final v13 ONNX model

The segment contains 261 processed frames and 207 frames with an accepted cup detection, giving 79.3 percent frame coverage. Stable frames reach approximately 0.79 to 0.85 confidence. The file is suitable as pose evidence, but it is not used as Jetson speed evidence because it is an offline replay.

The contact sheet also contains other black objects in the background. Some are detected as mice because actual mice are present on the desk. The cup claim is limited to the large green horizontal cup and does not imply that every object in the raw scene was part of the cup test.

22 ROS 2 Detection Publisher

The project now includes a ROS 2 Humble Python package named `yolo_detection_ros2`. The detector node opens a camera, runs the YOLO model, filters mouse and cup boxes with independent thresholds, publishes a `vision_msgs Detection2DArray` message, renders the live window and optionally writes the annotated video.

Each `Detection2D` message contains a bounding box centre, width and height, a text class ID and a floating point confidence score. The array header contains the camera frame name and ROS time. A receiving robot node can subscribe to `/yolo/detections` and make decisions without reading pixels or parsing console output.

```
cd ~/yolo_ros2_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash
ros2 launch yolo_detection_ros2 detector.launch.py

ros2 topic info /yolo/detections
ros2 topic echo /yolo/detections --once
ros2 topic hz /yolo/detections
```

I verified the package with `colcon build` in Ubuntu 22.04, and ROS 2 lists the `detector_node` executable. A WSL replay produced a cup message at 0.8217 confidence and about 35 Hz. The final hardware run then started the same package on `nvidia-desktop` with `/dev/video0`. Its saved topic record reports one publisher of type `vision_msgs/msg/Detection2DArray`, a mouse box at 0.9478 confidence and a measured rate of about 20.57 Hz.



Figure 9 Final Jetson camera run with detection overlay and ROS 2 publishing enabled

```
bash scripts/capture_jetson_ros2_evidence.sh 0 models/v13/best.pt ~/yolo_ros2_ws
# Portable proof: release/experiment_one_v13_ros2_jetson_evidence.zip
```

23 Saving Results and Typical Errors

The experiment saves three useful forms of evidence. Annotated AVI files show the visible output expected by the teacher. Raw AVI files preserve the original camera frames for later replay. Review directories contain contact sheets, selected still frames, CSV scoring tables and JSON run summaries. Together they support both demonstration and diagnosis.

Artifact type	Example	Purpose
Annotated Jetson video	v13 Jetson camera1 detected mouseCUP avi	Two class live evidence
Horizontal supplement	v13 horizontal cup success take1 avi	Successful flat cup pose
Raw recording	v13 pc camera1 raw files	Reproducible model replay
Scoring table	v13 20 angle results with horizontal cup csv	Auditable 18 of 20 result
Evidence sheet	v13 20 angle evidence with horizontal cup jpg	Visual summary
Run summary	v13 horizontal cup success take1 json	Thresholds frame counts and source

The two missed mouse views are typical false negative cases. Earlier versions also produced false positives on phones and black equipment. Those error types were converted into v11 and v12 training batches. This closes the loop between testing and dataset improvement and provides a repeatable procedure for future model versions.

24 Discussion of Failure Cases

The remaining misses are physically plausible. In an extreme side profile the mouse becomes a narrow dark strip with little visible wheel or button structure. In an underside view the sensor panel and feet look unlike the top surface that dominates ordinary mouse datasets. At 768 pixels the object is large enough, but the learned visual features are weak for those shapes.

A high 0.75 threshold reduces false positives but also rejects uncertain true objects. Lowering the threshold would recover some missed angles, yet it could reintroduce phone and chair errors. The current value therefore reflects the assessment setting: a conservative box is preferable to many incorrect mouse boxes. A better solution is more real side and underside training images, not simply changing the threshold after seeing the answer.

The validation plot also warns against overtraining the small specialised batch. More epochs would not necessarily fix the difficult views. I would collect several new mice with different shapes, keep whole scenes separated between train and test, and repeat the twenty angle protocol without selecting timestamps from the same training capture.

- Add extreme side and underside mouse images from at least three backgrounds.
- Add reflective and stainless cups lying on their sides.
- Keep phone and black equipment negatives in every new training export.
- Compare 0.65 0.70 and 0.75 thresholds on a fixed video before changing deployment defaults.

25 Reproducibility Guide

25 1 Audit and Export

```
py -3.13 scripts/export_audited_yolo.py `
--audit dataset_work/audit_dataset_v13 `
--output dataset_work/audit_dataset_v13/yolo_export
```

25 2 Train

```
wsl -d Ubuntu-22.04 bash -lc "cd /mnt/f/PycharmProjects/robomaster && bash
scripts/train_v13_horizontal_cup_wsl.sh"
```

25 3 Export ONNX

```
yolo export model=runs/detect/mouse_cup_yolo11n_v13_horizontal_cup_768/weights/best.pt format=onnx imgsz=768
simplify=True
```

25 4 Windows Camera Test

```
py -3.13 scripts/live_camera_onnx.py --camera 1 --model
runs/detect/mouse_cup_yolo11n_v13_horizontal_cup_768/weights/best.onnx --mouse-conf 0.75 --cup-conf 0.75 --
save results/v13_detected.avi --save-raw results/v13_raw.avi
```

25 5 ROS 2 Build

```
mkdir -p ~/yolo_ros2_ws/src
cp -r ros2/yolo_detection_ros2 ~/yolo_ros2_ws/src/
cd ~/yolo_ros2_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
```

25 6 ROS 2 Runtime Verification

```
bash scripts/capture_jetson_ros2_evidence.sh 0 /home/nvidia/jetson_yolo/models/v13/best.pt
/home/nvidia/yolo_ros2_ws
# Verify on the PC:
py -3.13 scripts/audit_experiment_one_submission.py --verify-git --require-jetson-ros2
```

26 Submission Artifact Inventory

Deliverable	Recommended file or directory
Dataset	dataset_work audit_dataset_v13 yolo_export
PyTorch model	runs detect mouse_cup_yolo11n_v13_horizontal_cup_768 weights best.pt
ONNX model	runs detect mouse_cup_yolo11n_v13_horizontal_cup_768 weights best.onnx
Windows program	scripts live_camera_onnx.py
Jetson program	scripts live_camera_pt.py
ROS 2 package and proof	ros2 yolo_detection_ros2 and release experiment_one_v13_ros2_jetson_evidence.zip
Jetson videos	results v13 Jetson camera1 detected files
Horizontal cup video	results v13_horizontal_cup_success_detected.avi
Twenty angle score	results v13_submission_review v13_20_angle_results_with_horizontal_cup.csv
Report	docs Experiment_One_Object_Detection_Report.docx and pdf

Large datasets and model weights may exceed ordinary GitHub limits. The repository should contain scripts, manifests, small evidence images, CSV summaries, README instructions and the report. Large weights and videos can be submitted as release assets or as a separate course archive when required. The report lists their exact local paths and hashes so the assessor can verify the files did not change.

27 Integrity Record

The following SHA 256 values were calculated from the final local files. They provide a simple integrity record for copying the submission to Jetson, cloud storage or removable media.

File	Bytes	SHA 256
v13 best.pt	5459930	14b82fb5b50d8140cebb134f6eb1d57e07902e07a78fbd5d033ab71d9025792
v13 best.onnx	10678826	42c78972a3a4f7f2a331df22a7758e9140a38d6736735391bc910cbf56f1aea0
horizontal cup video	12934534	7168ea036a9bc02b741103499d5ea843bc2277edb39199b9fc7cbf25da6cf231
Jetson ROS 2 evidence ZIP	12133767	8fe7bf7b4a57bf58e27676fd062099a4606ca87459e3eefaa75485e31a088e75

A checksum does not prove accuracy, but it prevents accidental substitution of a different model after the report is written. It is especially useful here because several version folders contain files named best.pt.

28 Conclusion

The final v13 model can detect a mouse and a cup in the same camera frame, including a cup lying flat. The most useful part of the work was not changing the network; it was keeping the bad predictions, correcting the labels and collecting the missing poses. The phone negatives and the horizontal-cup batch came directly from problems seen during live testing.

In the twenty-angle check, 18 scenes passed and 2 mouse views failed, giving 90 percent. The selected Jetson frames run from 15.7 to 21.2 FPS, above the required 5 FPS. The saved videos show the class, box, confidence and FPS, and the program can also keep the raw stream for later replay.

The ROS 2 package built and ran on the Jetson. Its stored topic output contains a mouse detection at 0.9478 confidence and a measured publish rate of about 20.57 Hz. The two missed mouse views are still included in the evidence. If I continue the project, the next data batch should focus on extreme side and underside views rather than adding more ordinary top views.

Appendix A Key Source Code Responsibilities

File	Responsibility
scripts live_camera_onnx.py	Windows camera input ONNX preprocessing NMS smoothing display and recording
scripts live_camera_pt.py	Jetson camera input Ultralytics CUDA inference display and recording
scripts run_video_onnx.py	Offline replay of a selected raw segment with JSON summary
scripts train_v13_horizontal_cup_wsl.sh	Repeatable v13 training command and hyperparameters
scripts export_audited_yolo.py	Only exports explicitly assigned train validation and test records
ros2 yolo_detection_ros2 detector_node.py	Publishes boxes class IDs and confidence as vision_msgs

The live inference file separates small functions such as letterbox and postprocess from the camera loop. This made offline video replay possible without copying the mathematical code. The ROS 2 node uses Ultralytics results directly because Jetson already has that framework installed. Both paths keep the same class order and deployment thresholds.

Appendix B Final Evidence Paths

```
results/v13_submission_review/v13_20_angle_evidence_with_horizontal_cup.jpg
results/v13_submission_review/v13_20_angle_results_with_horizontal_cup.csv
results/v13_horizontal_cup_success_detected.avi
results/v13_horizontal_cup_success_detected.json
results/v13_Jetson_camera1_detected_mouseCUP.avi
release/experiment_one_v13_ros2_jetson_evidence.zip
release/experiment_one_v13_video_evidence.zip
release/experiment_one_v13_model.zip
```

The older v13_Jetson_camera1_detected_CUP3.avi file is not part of the final proof. It remains in the working results directory only as historical test material. The curated CSV and evidence sheet do not reference it.

This report was generated from the current local workspace so that tables and integrity hashes match the files submitted with the experiment.